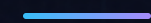


ENGENHARIA DE SOFTWARE II · 2026

RANK-IA

Sua organização de maneira eficiente



Integrantes

Fernando Hiroshi Murusaki · RA 241025851

Igor dos Reis Gomes · RA 241025265

Matheus Santos Magro · RA 231025335

Murilo Tomaz Gonzaga · RA 241024684

Grupo

Grupo 8

UNESP Bauru · BCC

O QUE É O RANK-IA?

Formação inteligente de equipes corporativas

Aplicação web corporativa que utiliza Inteligência Artificial para montar equipes otimizadas com base em habilidades, competências e gaps dos colaboradores.

- Geração automática de equipes com Coeficiente de Aproveitamento
- Ajustes manuais que retroalimentam o modelo de IA
- Sugestão de treinamentos baseados em gaps identificados
- Relatórios de desempenho individual e coletivo

PERFIL

Gestor / RH

Acesso total: cadastro, config. da IA, validação e relatórios

PERFIL

Colaborador

Consulta de competências, treinamentos e equipes

METODOLOGIA

Scrumban

Scrum + Kanban · Sprints de 2 semanas

ARQUITETURA DE SOFTWARE

Modelo C4

Três níveis de abstração para descrever a arquitetura da Plataforma RANK-IA

Nível 1 — Contexto

Nível 2 — Contêineres

Nível 3 — Componentes

C4

DIAGRAMA C4 — NÍVEL 1

Diagrama de Contexto

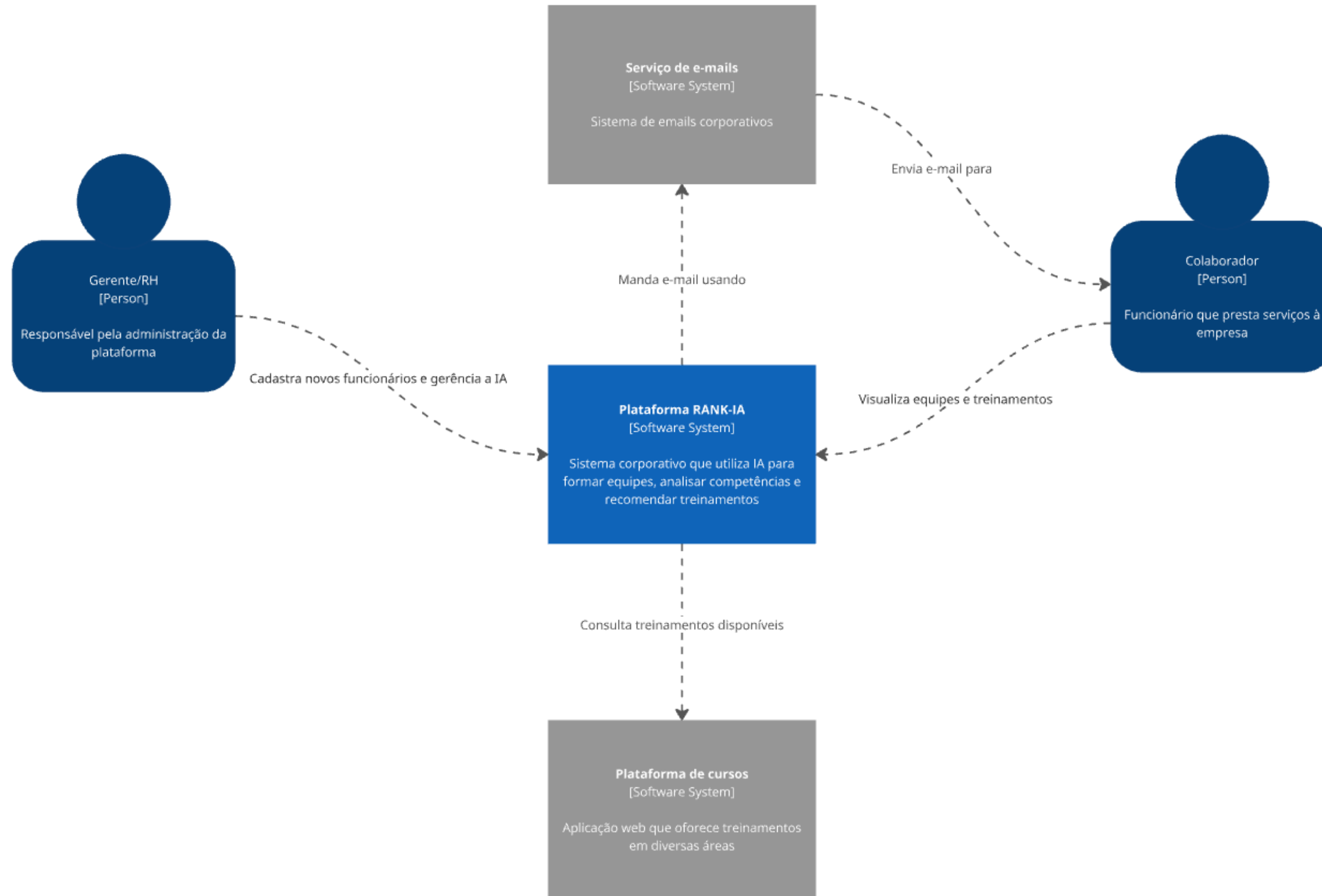


Diagrama de Contexto do Sistema para o RANK-IA

O diagrama de contexto do sistema para a plataforma RANK-IA

Última modificação: 2026-03-24

DIAGRAMA C4 — NÍVEL 2

Diagrama de Containers

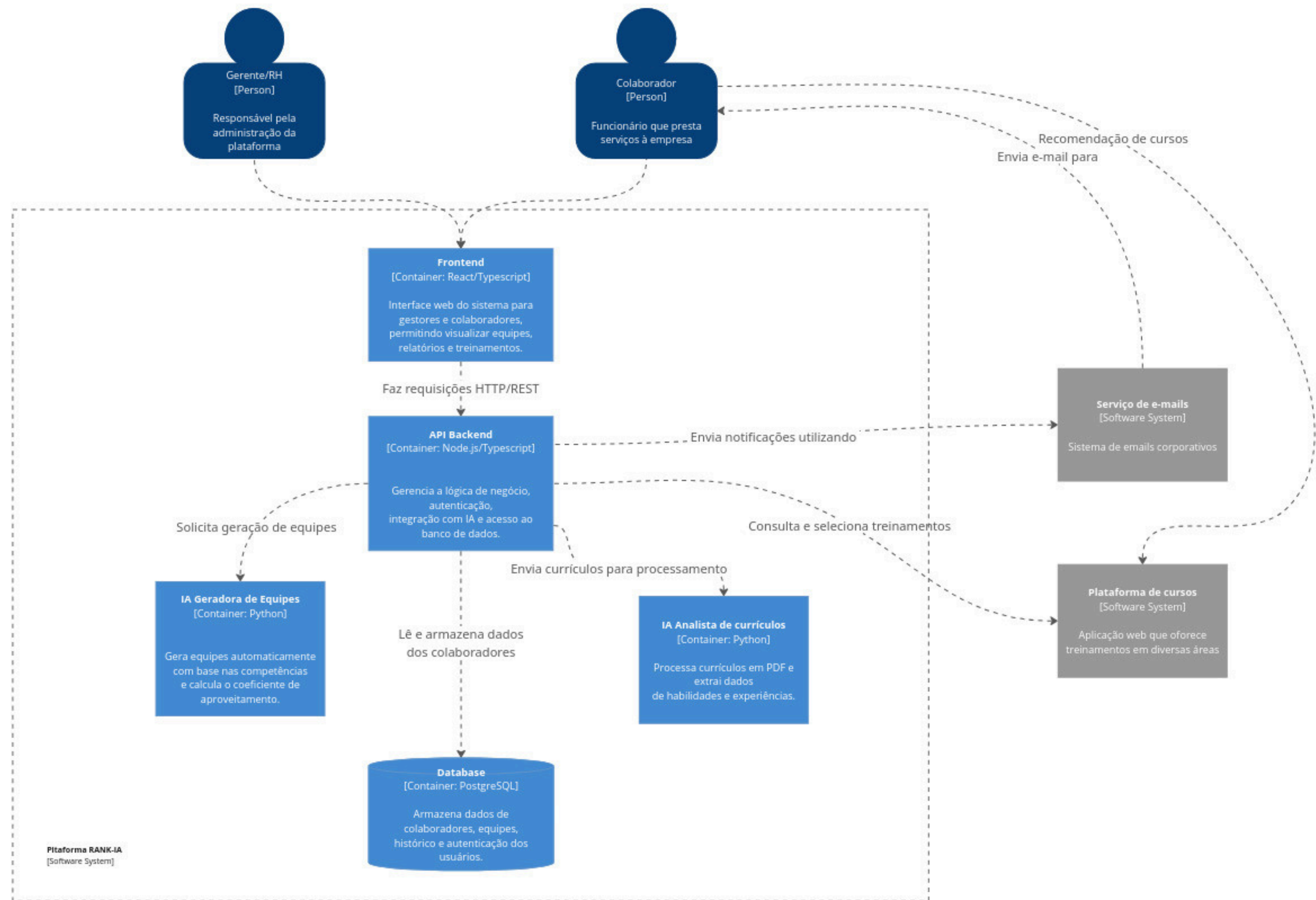
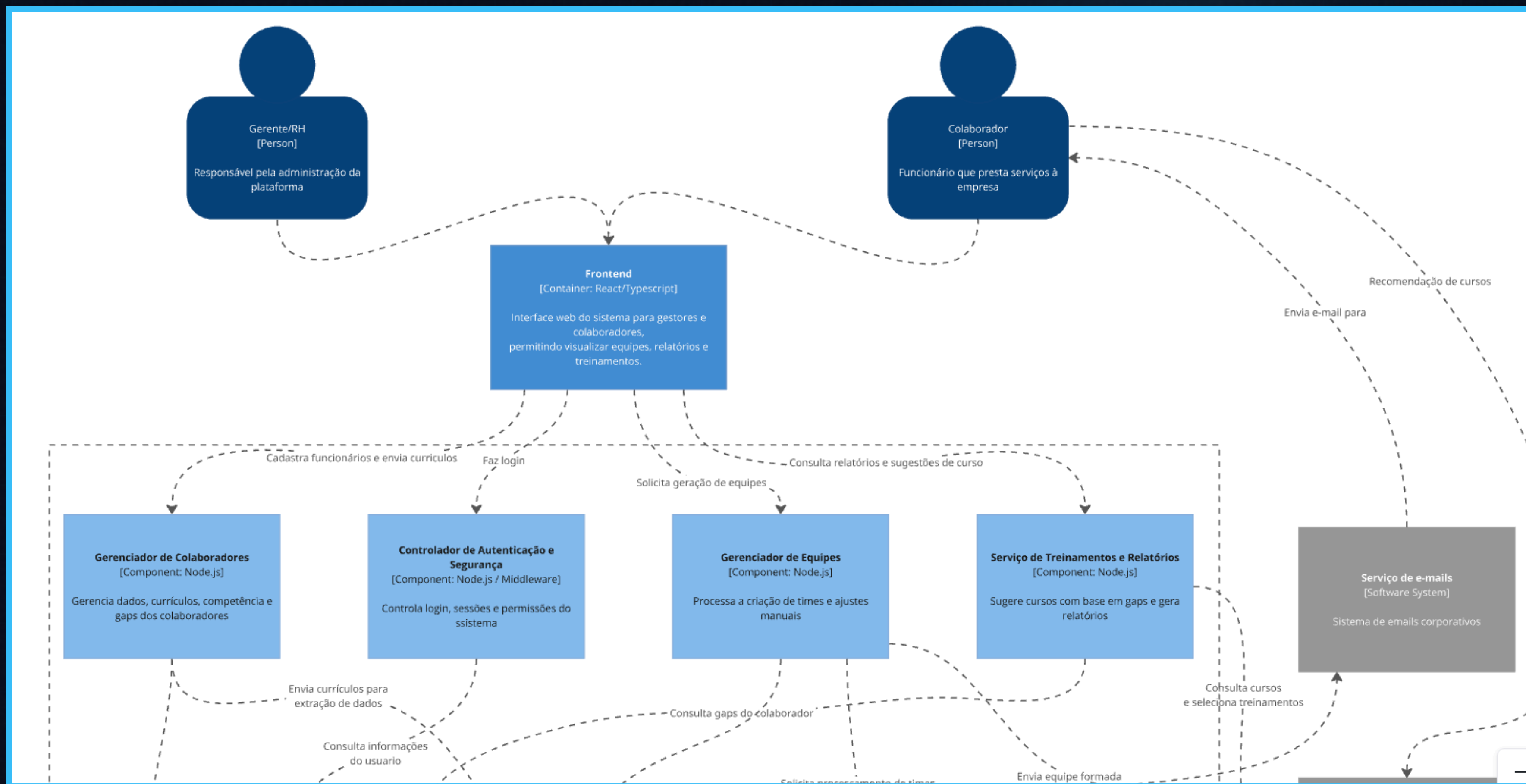


DIAGRAMA C4 — NÍVEL 2

Diagrama de Componentes



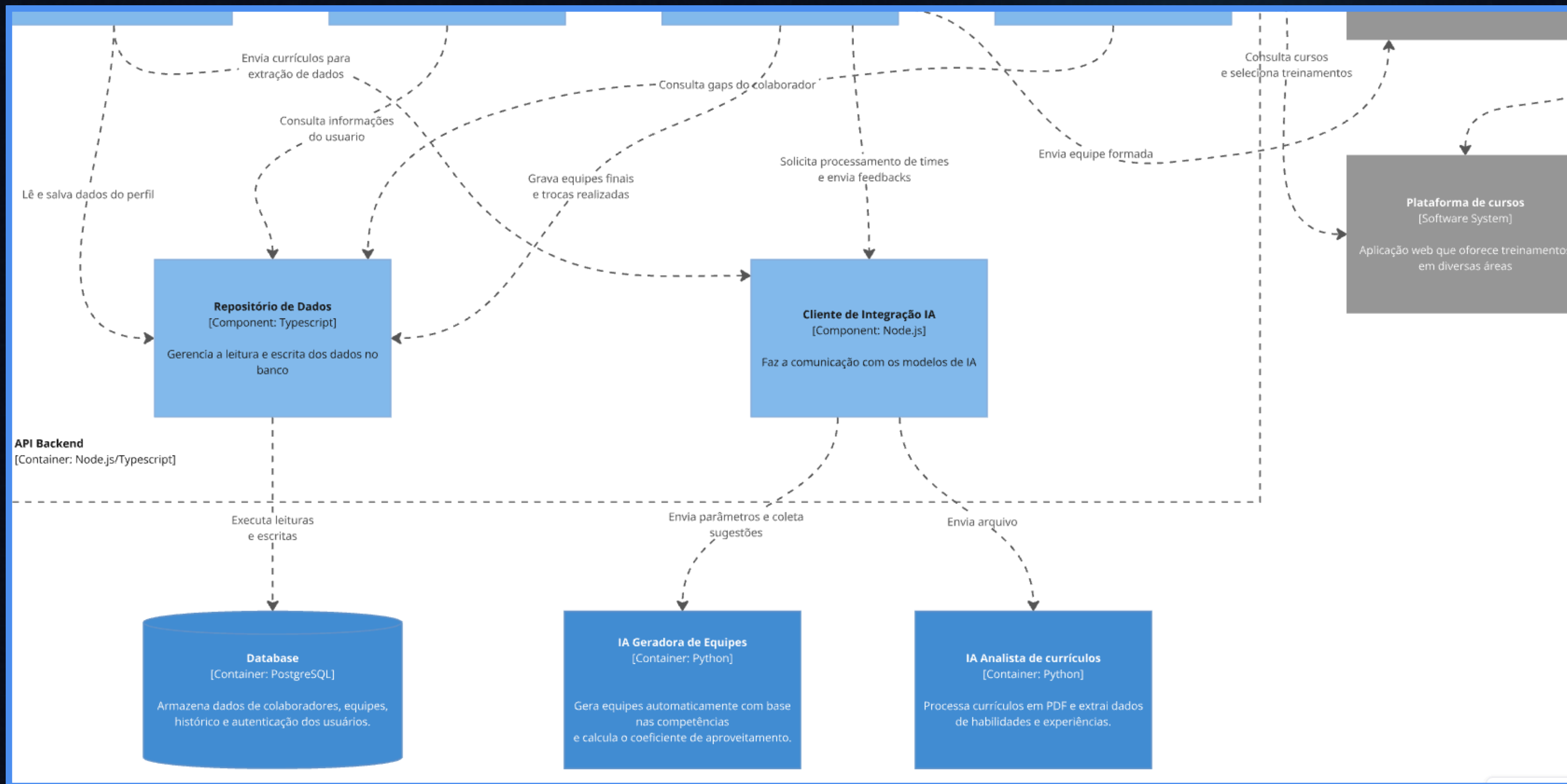


DIAGRAMA DE CLASSES

Gerenciador de Equipes

Modelagem UML do componente de gerenciamento de equipes da API Backend (Node.js/TypeScript)

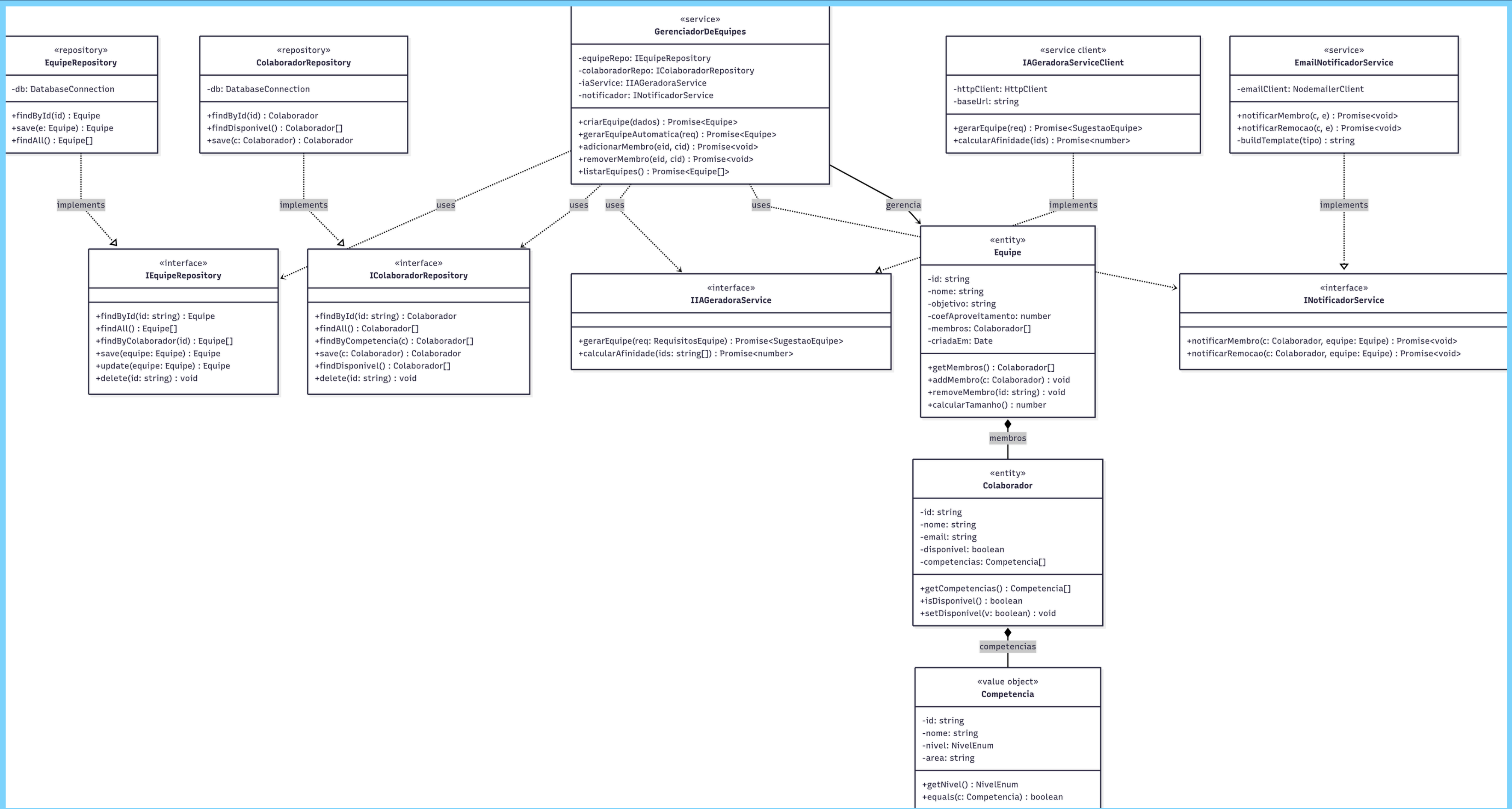
Entidades

Interfaces

Repositórios

UML

Diagrama de Classes — Gerenciador de Equipes



EMPACOTAMENTO

- **Organização por Domínio de Negócio:**
 - Módulo centralizado: `src/equipes/`
 - Alta coesão entre Entidades, Serviços e Repositórios
- **Aplicação do CCP (Common Closure Principle):**
 - Regra: *"O que muda junto, é empacotado junto"*
 - Mudanças na formação de equipes afetam apenas o módulo `equipes`
- **Respeito ao REP (Reuse/Release Equivalence Principle):**
 - `Equipe`, `Colaborador` e `GerenciadorDeEquipes` formam um conjunto lógico e indivisível
 - Versionamento e liberação unificados para os clientes da API

```
src/  
  
└─ equipes/ ← pacote centralizado  
  
    └─ entities/  
  
        └─ Equipe.ts  
  
        └─ Colaborador.ts  
  
        └─ Competencia.ts  
  
    └─ interfaces/  
  
        └─ IEquipeRepository.ts  
  
        └─ IIAGeradoraService.ts  
  
        └─ INotificadorService.ts  
  
    └─ services/  
  
        └─ GerenciadorDeEquipes.ts  
  
└─ repositories/  
  
    └─ EquipeRepository.ts
```


PRINCÍPIO SOLID

Dependency Inversion Principle

"Módulos de alto nível não devem depender de módulos de baixo nível.

Ambos devem depender de abstrações."

D

Dependency Inversion

Classes de **alto nível** (como `GerenciadorDeEquipes`) não dependem de implementações concretas — apenas de **interfaces/abstrações**.

Aplicado em: API Backend Node.js/TypeScript)

DIP

Acoplamento direto a implementações concretas

✗ *GerenciadorDeEquipes acoplado a classes concretas*

```
import { EquipeRepository } from './EquipeRepository'
import { ColaboradorRepository } from './ColaboradorRepository'
import { IAGeradoraServiceClient } from './IAGeradoraServiceClient'
import { EmailNotificadorService } from './EmailNotificadorService'
```

```
class GerenciadorDeEquipes {
  private equipeRepo: EquipeRepository
  private colaboradorRepo: ColaboradorRepository
  private iaService: IAGeradoraServiceClient
  private notificador: EmailNotificadorService

  constructor() {
    // instancia direto - sem injeção
    this.equipeRepo = new EquipeRepository()
    this.colaboradorRepo = new ColaboradorRepository()
    this.iaService = new IAGeradoraServiceClient()
    this.notificador = new EmailNotificadorService()
  }

  async gerarEquipeAutomatica(req: any) {

    const sugestao = await this.iaService.gerarEquipe(req)

  }
}
```

Problemas

- Impossível trocar implementação do BD sem alterar GerenciadorDeEquipes
- Testes unitários exigem banco real ou serviço IA real
- Trocar e-mail por SMS obriga edição da classe de negócio
- Alto acoplamento = baixa manutenibilidade

Módulo de alto nível (negócio) depende de módulos de baixo nível (infraestrutura)

✓ Com DIP Injeção de dependências via interfaces

```
✓ Depende apenas de interfaces (abstrações)
interface IEquipeRepository {
  findById(id: string): Promise<Equipe>
  save(e: Equipe): Promise<Equipe>
  findAll(): Promise<Equipe[]>
}

interface IIAGeradoraService {
  gerarEquipe(req: RequisitosEquipe): Promise<SugestaoEquipe>
}

class GerenciadorDeEquipes {
  constructor(
    private equipeRepo: IEquipeRepository,      ← interface
    private colaboradorRepo: IColaboradorRepository, ← interface
    private iaService: IIAGeradoraService,      ← interface
    private notificador: INotificadorService    ← interface
  ) {}

  async gerarEquipeAutomatica(req: RequisitosEquipe) {
    const sugestao = await this.iaService.gerarEquipe(req)
    const equipe = Equipe.fromSugestao(sugestao)
    const saved = await this.equipeRepo.save(equipe)
    for (const membro of saved.getMembros())
      await this.notificador.notificarMembro(membro, saved)
    return saved
  }
}
```

Benefícios

- ✓ Fácil troca: PostgreSQL → MongoDB sem alterar lógica
- ✓ Testes com mocks/stubs das interfaces
- ✓ E-mail substituído por SMS/Push sem tocar negócio
- ✓ Classes de negócio isoladas de infraestrutura

Uso com framework NestJS

```
@Injectable()
export class GerenciadorDeEquipes {
  constructor(
    @Inject('IEquipeRepository')
    private repo: IEquipeRepository
  ) {}
}
```

PRINCÍPIO SOLID — 2º EXEMPLO

Single Responsibility Principle

"Uma classe deve ter um, e somente um,
motivo para ser modificada."

SRP

✗ Sem SRP

GerenciadorDeEquipes com múltiplas responsabilidades

```
// ✗ Uma única classe faz tudo
class GerenciadorDeEquipes {
  private db: DatabaseConnection

  async criarEquipe(dados: any) {
    // Resp. 1: lógica de negócio
    const equipe = new Equipe(dados)

    // Resp. 2: persistência — deveria ser do Repository
    const sql = `INSERT INTO equipes VALUES ($1,$2)`
    await this.db.query(sql, [equipe.id, equipe.nome])

    // Resp. 3: envio de e-mail — deveria ser do Notificador
    const transport = nodemailer.createTransport({...})
    await transport.sendMail({ to: membro.email, ... })

    // Resp. 4: chamada à IA — deveria ser do IAServiceClient
    const res = await fetch('http://ia-service/gerar',{...})
    const sugestao = await res.json()

    return equipe
  }
}
```

4 responsabilidades numa classe

- 1 Lógica de negócio (equipes)
- 2 Persistência no banco de dados
- 3 Envio de notificações por e-mail
- 4 Comunicação com o serviço de IA

Qualquer mudança em persistência, e-mail ou IA obriga a editar a classe de negócio — alto risco de regressão

Cada classe com uma única responsabilidade

```
// ✓ GerenciadorDeEquipes só coordena (resp. 1)
class GerenciadorDeEquipes{
  constructor(
    private repo: IEquipeRepository,      delega persistência
    private notif: INotificadorService,    delega notificação
    private ia: IIAGeradoraService        delega IA
  ) {}

  async criarEquipe(dados: DadosEquipe) {
    const sugestao = await this.ia.gerarEquipe(dados)
    const equipe = Equipe.fromSugestao(sugestao)
    await this.repo.save(equipe)           //Repository persiste
    for (const m of equipe.getMembros())
      await this.notif.notificarMembro(m, equipe)
    return equipe
  }
}

// Resp. 2: só persiste equipes

class EquipeRepository implements IEquipeRepository {
  async save(e:Equipe)
  async findById(id:string)
}

// Resp. 3: só envia notificações por e-mail
class EmailNotificadorService implements INotificadorService {
  asyncnotificarMembro(c: Colaborador, e: Equipe) {
    await this.mailer.send({ to: c.email, ... })
  }
}

// Resp. 4: só se comunica com a IA
class IAGeradoraServiceClient implements IIAGeradoraService {
  async gerarEquipe(req: RequisitosEquipe) {
    return fetch(this.baseUrl + '/gerar', { method: 'POST', ... })
  }
}
```

Benefícios do SRP

- ✓ Trocar PostgreSQL → MongoDB: só alterar EquipeRepository
- ✓ Trocar e-mail → SMS: só alterar EmailNotificadorService
- ✓ Trocar modelo de IA: só alterar IAGeradoraServiceClient
- ✓ Testes unitários isolados por responsabilidade

SRP + DIP juntos:

Classesde responsabilidade única + injeção via interfaces = código modular, testável e fácil de evoluir

RESUMO

Conclusão

- C4** Arquitetura modelada em 3 níveis: **Contexto**, **Contêineres** e **Componentes**, permitindo visão clara de cada camada do sistema.
- UML** Diagrama de classes do **Gerenciador de Equipes** com entidades, interfaces e implementações bem definidas.
- D** **DIP** - injeção via interfaces
SRP - Uma responsabilidade por classe. Código modular, testável e fácil de evoluir.

STACK

Frontend React / TypeScript

Backend Node.js / TypeScript

IA Python (2 serviços)

Banco PostgreSQL

Grupo 8 · Engenharia de Software II · 2026/1